OPG, Final Minimal Design

Necessary-and-Sufficient Models, Metrics, Assumptions, and Protocol for P1

Consolidated revision for `train_gpt.py`

2026-06-04

Contents

1 Goal and Claim Hierarchy

The project should first establish the smallest decisive claim: under a fixed memory budget, recurrent depth buys task utility on data that actually requires depth. The claims are ordered so that later mechanisms cannot substitute for a failed primary result:

- **P1, primary:** $G_T(K_{\text{lo}}, K_{\text{hi}}) > 0$ at fixed memory on a depth-hard task, with a passing positive control.
- **P2, memory:** reversible reconstruction makes activation memory approximately constant in recurrent depth K. This claim is largely severable from P1 and must be measured independently.
- **P3, secondary:** an MoE basis recovers the depth diversity lost by weight tying. This claim is staged only after P1 holds.

The design serves P1. P2 verifies the training-memory niche. P3 is optional capacity recovery, not the premise of the first experiment. The core *retains* the mechanisms that directly advance the two design goals: MLA (low-rank joint KV compression), smooth-sparse MoE (no hard top-k), and a mixture-of-softmaxes (MoS) output head are retained for the resource (MLA low-rank KV; smooth-sparse MoE compute) and expressiveness (MoE effective-depth basis, MoS output-rank) goals. What it removes are feature stacks that obscure P1 failure attribution rather than advancing the goals: Dirichlet–UCB routing, implicit/DEQ-style backward, fixed-point consistency, spectral or Lyapunov pressure, UFID, distillation, and anytime losses are excluded from the P1 science path. The executable pipeline is only the positive-control, M_0, and M_{clk} triage. MoE, cache, and deployment axes are executable only through a separate staged diagnostic harness. This preserves the causal split: S+0 promotes or fails P1, while S+1/S+2/S+3 report mechanism and deployment diagnostics without becoming explanations for a failed P1 gate.

2 Models: The Necessary Set

Core M_0. The core model is a tied, token-injected, additive-coupling reversible recurrence whose F_θ and G_θ blocks are pre-norm MLA attention [**?**] plus a smooth-sparse SwiGLU mixture-of-experts (MoE). The recurrent state is $s_k = (a_k, b_k)$, with input embedding state x_0:

$$a_{k+1} = a_k + F_\theta(\text{RMSNorm}(b_k + x_0)) \,, \tag{1}$$

$$b_{k+1} = b_k + G_\theta(\text{RMSNorm}(a_{k+1} + x_0)) \,. \tag{2}$$

The inverse is structural and floor-free:

$$b_k = b_{k+1} - G_\theta(\mathrm{RMSNorm}(a_{k+1} + x_0)) \,, \tag{3}$$

$$a_k = a_{k+1} - F_\theta(\mathrm{RMSNorm}(b_k + x_0)) \,. \tag{4}$$

Invertibility is therefore an algebraic property of the additive coupling, not a consequence of contraction. The core does not require a fixed point.

The readout is simple: for a recurrent budget K, the final recurrent state

$$z_K = \tfrac{1}{2}(a_K + b_K)$$

is fed to a mixture-of-softmaxes (MoS) output head [**?**], which mixes a small number of softmax components over the tied embedding basis to break the softmax-rank bottleneck on the output distribution. There is no halting gate: depth is selected by the sampled/evaluated recurrent budget K, and the recurrence's F_θ/G_θ blocks use MLA attention and smooth-sparse SwiGLU MoE, trained with full reversible BPTT [**?**, **?**] ($O(1)$ activation memory in K, §**??**) so deeper inference is not penalized by activation storage. The MoE router is a control-experiment knob, `router_type`$\in \{$`relu`, `softmax`$\}$ (ReLU routing per ReMoE [**?**] is continuous, reconstruction-stable, and reversibility-safe; cf. MoEUT [**?**]). The trained model can still fail; the point is that the architecture composes shared low-rank experts per step into high *effective* depth. Nestedness across depth is a property of the additive-coupling recurrence and budget selection, not of a halting gate.

The memory-critical target implementation uses reversible full BPTT through K_{hi}. Truncation is only a calibrated fallback, because full reversible BPTT is the core advantage for P2. The Tier-1 executable harness may use ordinary stored-activation autograd for detectability and structural-inverse tests; it must not claim P2 activation-memory scaling until the memory-saving backward path is implemented and measured. The P1 loss is:

$$\mathcal{L} = \mathbb{E}_{K_{\mathrm{hi}} \sim q_{\mathrm{deep}}}[L_T(K_{\mathrm{hi}})] + \lambda_h \left[L_T(K_{\mathrm{hi}}) - \mathrm{sg}(L_T(K_{\mathrm{lo}})) + m\right]_+ , \qquad \lambda_h \text{ small.} \tag{5}$$

Hard NaN/Inf guards and a light boundedness penalty,

$$[\|z_k\|_{\mathrm{RMS}} - B_z]_+^2,$$

are stability guards, not fixed-point losses.

Implementation note (Tier-1 harness). The executable Tier-1 harness (`experiments/p1_synthetic.py`) currently optimizes only the $L_T(K_{\mathrm{hi}})$ cross-entropy term; it does *not* yet apply the λ_h no-degradation hinge or the boundedness penalty above. Those terms are realized in the `train_gpt.py` finite-horizon path. The S+0 detectability results are therefore from the plain-L_T objective.

Clock diagnostic M_{clk}. M_{clk} is M_0 with an explicit step embedding e_k injected into *both* half-step updates each iteration:

$$a_{k+1} = a_k + F_\theta(\mathrm{RMSNorm}(b_k + x_0 + e_k)) \,, \qquad b_{k+1} = b_k + G_\theta(\mathrm{RMSNorm}(a_{k+1} + x_0 + e_k)) \,,$$

where e_k is sinusoidal or learned. This model is the upper bound on depth-conditioned computation. It is diagnostic only and excluded from the final model unless its extrapolation cost is measured and accepted.

Positive control. The harness must include a vanilla looped-transformer positive control: input-injected, recurrent depth exposed, activations stored rather than reconstructed, and otherwise deliberately ordinary. If this model does not scale on the Tier-1 depth-hard task, the harness, task, or scoring is broken.

Future ablation models. M_0 is the tied non-MoE control. M_{MoE} uses a plain softmax or sparse top-r router and marginal-load balance only for S+1; M_{static} freezes routes across depth. M_{nonrev} stores activations and isolates the reversible-memory benefit. These are not part of the current P1 executable interface. Their code path lives in `experiments/p1_feedback_stages.py` and is diagnostic until the positive control and M_0 make depth utility detectable.

Explicit exclusions from P1. The following are not part of the core science path: Dirichlet–UCB router, int6 quantization, DEQ or implicit backward, fixed-point consistency, prefix-anchor consistency, spectral or Lyapunov pressure, UFID, distillation, and anytime losses. They may remain in the repository as legacy, deployment, or later-stage axes, but they should not be used to establish P1. MLA, smooth-sparse MoE, and the MoS output head are *retained* in M_0 for the resource and expressiveness goals (§**??**, §**??**); the MoE router type is a control-experiment knob, not an excluded mechanism.

3 Experimental Protocol

Tier 1: detectability on synthetic depth-hard tasks. The primary task is iterated S_5 permutation composition, with sequence length ℓ tuning the required depth. Barrington's theorem shows that bounded-width polynomial-size branching programs recognize exactly nonuniform NC^1, and the S_5 permutation construction is the canonical reason this task is a principled depth probe [**?**]. It is not a language-modeling benchmark; it is the depth-demand smoke test. Alternates are parity as a sanity floor and s-t graph reachability with tunable diameter.

Run M_0, the vanilla positive control, and M_{clk} through the identical harness. The purpose is to confirm that $G_T > 0$ is detectable and to localize failure before spending compute on small language modeling. The in-repository executable harness is `experiments/p1_synthetic.py`; its P1-promotion variants are `control`, `m0`, and `mclk`. The same runner deliberately rejects MoE, cache-slope, and int6 switches. That small interface is part of the experiment: a failed positive control should trigger task, budget, or block-capacity debugging, not another mechanism. The reproducible DDP pipeline is `experiments/run_p1_synthetic_pipeline.sh`. The separate feedback-stage runner `experiments/run_feedback_stage_pipeline.sh` composes the S+0 run with later-stage diagnostics; its outputs are not P1-promotion evidence.

Tier 2: deployment relevance. After Tier 1 passes, evaluate small language modeling on byte/char-level `text8` or `enwik8`, or on TinyStories. Report BPB where natural, and compare against a matched small dense transformer on the Pareto frontier.

Curriculum. Use Huginn-style recurrence-count sampling for q_{deep} and test-time depth scaling, plus per-step input injection [**?**]. Do not borrow truncated BPTT for the core: reversibility affords full BPTT and is the memory advantage being tested.

Scale and statistics. Use $d_{\text{model}} \in [256, 512]$, one reversible block, context length at most 1024, and multiple seeds. Evaluate

$$K \in \{4, 8, 16, 32, 64, 128\}.$$

Report the convergence knee first, then choose $(K_{\text{lo}}, K_{\text{hi}})$ to straddle it. Every headline G_T uses paired confidence intervals. The executable DDP harness emits a streaming paired normal-approximation interval from all-reduced sums and squared sums; offline reports may replace this with a paired bootstrap over saved per-example gains.

4 Metrics: Gates Versus Diagnostics

Metric	Role	Rule / mechanics
$G_T(K_{\text{lo}}, K_{\text{hi}})$	**GATE** (P1)	Paired CI, lower bound > 0, on a depth-hard task, w knee-straddling pairs and a passing positive control.
NDR_ϵ	**GATE** (guard)	No-degradation rate $\leq \delta_{\text{task}}$.
$\text{VRAM}_{\text{peak}}$, slope(VRAM/batch), α_{act}, R_{rec}	**GATE** (P2, resource = memory-efficiency)	Peak VRAM is the primary resource number; VRAM-vs-batch scaling slope should be low/flat ($\text{slope}_{\text{MB/sample}}$, OLS fit `vram_vs_batch_scaling`); $R_{\text{act}}(K, 2K) \leq 1.10$ (activation memory ≈constant in recurrent depth K); reconstruction error acceptable u actual training stochasticity. Dropped as resource metrics: the artifact-16 MB hard gate (artifact bytes logged informationally only), FLOPs/token, and wall-clock/step. The active-expert fraction is demote a MoE-mechanism diagnostic (`diag:active_frac`), n resource gate.
Pareto (S_T, VRAM, ms/token)	**GATE** (sufficiency)	On or above frontier versus a matched small dense transformer. This, not $G_T > 0$ alone, is project succe
Recurrence-equivalence exponent	**DIAG** (headline)	K recurrent loops are worth how many untied layers; on the synthetic task where hardness is known.
Convergence-depth $k^\star(x)$ vs. hardness	**DIAG**	Correlate with ℓ or graph diameter. Flat and small k the plateau signature.
WFIR	**DIAG**	Use task-aligned readout, such as correct-token log-probability or NLL contribution; do not use rando projections as the headline. Report $\text{WFIR}_{\text{real}} - \text{WFIR}_{\text{shuffled}}$.
Δ_{MoE}, AEBR, route-depth NMI	**DIAG** (future P3)	Use only after P1 passes and only in a separate MoE ablation. They are not emitted by the P1 harness.
ρ_F, residual, $\sigma_{\max}(J_F)$	**DIAG**	Convergence diagnostics only; feed $k^\star$ and fallback decisions.
$\text{Gap}_{\text{dense}}$, $\text{KL}_{\text{teacher}}$	**DIAG**	Dense simulation is the weakest claim and never a ga

Useful function-space depth. For a fixed probe set X, let $f_k(X)$ be a task-aligned readout at recurrent depth k. Define increments:

$$\Delta_k(X) = f_{k+1}(X) - f_k(X).$$

WFIR is the effective rank of centered, regularized functional increments, reported against a shuffled-depth baseline. It is diagnostic because diversity alone can be noise; it becomes meaningful only when paired with positive G_T and controlled NDR_ϵ. Legacy internal metrics such as `ED_update`, `ED_logit`, `route_depth_nmi_mean`, `route_depth_nmi_max`, and `expert_util_mean` map to older ED_u, ED_ℓ, route-depth NMI, and expert-utilization traces. They are useful for failure analysis, but they are not P1 gates.

5 Clock Diagnostic and Failure Triage

Run M_0, the vanilla control, and M_{clk} on Tier 1. Interpret outcomes as follows:

Observation	Localized cause / action
Control does not scale on a provably depth-hard task	Harness or metric is broken: pairs may not straddle the knee, scoring may be wrong, or the task is too easy. Fix this before model work.
Control scales; M_0 scales	Core is sound. Proceed to Tier 2, then optional P3 work.
Control scales; M_0 does not; M_{clk} scales	Bottleneck is autonomous depth signal, such as vanishing displacement. Pursue sustained dynamics, velocity-state routing, or accept a mild clock with measured extrapolation cost.
Control scales; neither M_0 nor M_{clk} scales	Bottleneck is block expressivity or parameterization, not the signal. Repair the block, not the router.
M_{clk} raises G_T, but extrapolates poorly	Clock works as an upper bound and its cost is extrapolation. This justifies investing in an autonomous substitute.

The clock is a diagnostic ceiling, not a design commitment. It answers whether depth-conditioned computation is useful on the data before investing in autonomous routing.

6 Assumptions and Checks

	Assumption	How it is checked, not assumed
A1	The task requires depth	Provable depth-hard synthetic task plus positive control. If neither control nor clock scales, the harness or block is the issue.
A2	Reversibility holds numerically	R_{rec} measured with real stochasticity; replay RNG state for stochastic operations in the recurrent path.
A3	Memory is binding and latency tolerable	Pareto gate versus a small dense model. If a small dense model wins, the memory-bound niche is absent.
A4	Nestedness holds without a halting gate	Depth is selected by the recurrent budget K; verify the depth-K_{lo} function is reachable inside the depth-K_{hi} class, otherwise nestedness is only approximate.
A5	Additive coupling stays bounded over K	Boundedness penalty plus hard guards; monitor norm growth as a stability and throughput cost.
A6	Optional P3 capacity recovery is needed	Tested only after P1 holds; otherwise capacity mechanisms are premature.

The following are demoted from assumptions to tested hypotheses: routes specialize without a clock, and dense soft routing is affordable. The latter is moot in the P1 core and re-enters only in S+1.

7 Staging

S+0. M_0 on Tier 1, then Tier 2, with the clock triage. Gate: P1 paired CI, P2 memory, and Pareto. Current executable: `experiments/p1_synthetic.py -variant m0`, compared against `-variant control` and `-variant mclk`.

S+1, P3. A separate M_{MoE} ablation uses plain or sparse routing and marginal-load balance only. Dirichlet–UCB stays deleted from the science path. Gate after S+0 passes: $\Delta_{\text{MoE}} > 0$ at matched budget; AEBR, route-depth NMI, and expert utilization are mechanism diagnostics. This stage is intentionally not wired into `experiments/p1_synthetic.py`.

S+2, KV axis. Terminal hidden-state cache (emitted in log contracts as `hidden_state_proxy_not_autoregress` with matched training including terminal-context attention. Compare exact multi-depth cache and shared-cache baselines. Gate: $\alpha_{\text{kv}} \approx 0$ and quality gap $\leq \tau_{\text{cache}}$. The staged synthetic runner emits only a hidden-state memory proxy and explicitly marks the autoregressive KV quality gap as untested; an autoregressive LM harness is required before claiming constant-KV correctness.

S+3, deployment. Add int6, MLA/GQA, sparse inference, and other deployment features. Gate: post-quant Pareto with quantization gap reported. The current staged runner implements a fake-int6 linear-weight sensitivity check only; it is outside the P1 harness.

Fallback. Fixed-point pressure only if S+0 fails by instability, severe overthinking, or extrapolation. Use the weakest sufficient stabilizer first.

8 Pipeline Simplification Audit

The active pipeline should have the smallest interface that can falsify or support P1. A mechanism is included only if removing it would make the P1/P2 question unanswerable. This gives the following first-principles pruning rule:

keep(m) $\iff$ m is necessary to test P1/P2 now, or is the positive-control/clock diagnostic needed to localize failu

Everything else is staged.

Pipeline surface	**First-principles critique**	**Current fix**
MoE, static-route MoE, expert-basis metrics	P3 mechanisms cannot rescue a failed P1 result and make failure attribution ambiguous.	Removed from the P1 harness; available only in the separate S+1 diagnostic runner.
Dirichlet–UCB, MLA, MoS, cache modes	Deployment/capacity features do not answer whether recurrence itself buys utility.	Dirichlet–UCB/MLA/MoS remain excluded; cache appears only as an S+2 proxy diagnostic.
Mean-only G_T reporting	A tiny positive mean can be noise and should not be promoted.	Pair rows now include G_{nll}, G_{acc}, and confidence intervals.
Hardcoded task budget in the shell runner	A failed positive control could be task hardness, insufficient iterations, or model failure; hardcoding prevents localization.	Runner exposes only task and budget knobs such as `SEQ_LEN`, `ITERATIONS`, `TRAIN_DEPTHS`, and `PAIRS`.
Legacy hypothesis queue	It still described the old RevDEQ/Parcae/MoE stack as active, steering agents toward refuted or non-P1 work.	The active snapshot is final-minimal P1; the older dense-MoE ledger is historical evidence.
Soft Lyapunov or Lipschitz-band pressure	It optimizes a diagnostic proxy rather than task utility and previously hurt validation.	Launch validation rejects nonzero `lyapunov_coef`; FP quantities remain diagnostics.

This audit leaves one intentionally small P1 promotion surface:

`control`, `m0`, `mclk`.

It has one job: determine whether depth utility is detectable at all. It should not grow options for P3, KV, or deployment while the positive-control gate is failing. The audit also leaves one known implementation gap: `experiments/p1_synthetic.py` verifies structural reversibility and P1 detectability, but it does not yet implement custom reversible activation reconstruction for training memory. That is intentional staging, not a P2 claim. P2 is promoted only after activation-memory slope and reconstruction error are measured in the actual memory-saving backward path.

8.1 Feedback-stage implementation split

The repository now has two executable surfaces:

- `experiments/p1_synthetic.py`: the minimal S+0/P1 harness. It accepts only `control`, `m0`, and `mclk`.
- `experiments/p1_feedback_stages.py`: non-core staged diagnostics. S+1 runs `m0`, `moe`, and `static_moe`; S+2 emits a hidden-state cache proxy labelled `hidden_state_proxy_not_autoregressive_kv`; S+3 emits fake-int6 K-sweep and pair diagnostics.

The full feedback-stage command is:

```
CUDA_VISIBLE_DEVICES=6,7 ITERATIONS=1000 RUN_TAG=gpu67_feedback_all_i1000 bash experiments/run_
```

This command is useful for executable coverage of the feedback document. Its S+1/S+2/S+3 rows are mechanism/deployment diagnostics, not substitutes for a passing S+0 positive-control gate.

9 Current Simplified S+0 DDP Verification

On June 4, 2026, after pruning the executable harness to the necessary P1 surface, the simplified S+0 pipeline was run with

`CUDA_VISIBLE_DEVICES=6,7 ITERATIONS=1000 RUN_TAG=gpu67_s0_pruned_i1000`.

The run used two A100 GPUs, $\ell = 16$, train depths $\{16, 32, 64\}$, eval depths $\{4, 8, 16, 32, 64\}$, and 4096 paired evaluation examples per depth pair.

Variant	Params	Peak MB	Best K	Best loss	$G_{\mathrm{nll}}(16,64)$ [95% CI]	$G_{\mathrm{acc}}(16,64)$ [95% CI]	NDR
`control`	198,521	1316.0	16	4.7930	−0.000477 [−0.000951, −0.000002]	−0.000488 [−0.000188, 0.001165]	0.5122
`m0`	363,897	2539.1	16	4.7934	0.000071 [−0.000270, 0.000412]	−0.000977 [−0.002767, 0.000814]	0.4968
`mclk`	429,433	2540.1	16	4.7932	−0.000073 [−0.000351, 0.000205]	0.000000 [0.000000, 0.000000]	0.5146

Variant	$G_{\mathrm{nll}}(8,32)$ [95% CI]	$G_{\mathrm{acc}}(8,32)$ [95% CI]	NDR	R_{rec}
`control`	−0.000203 [−0.001380, 0.000974]	−0.000732 [−0.001998, 0.000534]	0.5151	N/A
`m0`	0.000203 [−0.000573, 0.000979]	−0.000244 [−0.002098, 0.001609]	0.5039	9.57×10^{-6}
`mclk`	0.000658 [0.000027, 0.001289]	0.000000 [0.000000, 0.000000]	0.4917	1.12×10^{-5}

The verification validates the simplified interface, DDP operation, finite-loss training, CI-backed pair reporting, and structural reversibility. It does not promote P1. The positive control is negative on the main $16 \to 64$ NLL gain and NDR_ϵ remains near one half. M_{clk} shows a small positive $8 \to 32$ NLL interval, but the clock diagnostic cannot rescue the failing positive-control gate. Following the triage table, the next principled step is detectability repair: verify the target representation and positive-control capacity, use a shorter or curriculum-shaped composition task, or add a simpler positive control known to solve the task before testing M_0. Adding MoE, cache, quantization, or fixed-point pressure would not address this blocker.

10 Current Feedback-Stage DDP Verification

The separate feedback-stage pipeline was run on June 4, 2026 with

`CUDA_VISIBLE_DEVICES=6,7 ITERATIONS=1000 RUN_TAG=gpu67_feedback_all_i1000`.

It wrote current S+0 outputs plus S+1/S+2/S+3 diagnostic outputs under `experiments/training_logs/p1_feed`. This run verifies the code coverage requested by the feedback document while preserving the minimal P1 interface.

Run	Params	Peak MB	Best K	Best loss	$G_{\mathrm{nll}}(16,64)$ [95% CI]	NDR	R_{rec}
`s0_control`	198,521	1316.0	16	4.7930	-0.000477 $[-0.000951, -0.000002]$	0.0122	N/A
`s0_m0`	363,897	2541.0	16	4.7934	0.000071 $[-0.000270, 0.000412]$	0.4968	9.57×10^{-6}
`s0_mclk`	429,433	2540.1	16	4.7932	-0.000073 $[-0.000351, 0.000205]$	0.5146	1.12×10^{-5}
`s1_m0`	363,897	2539.1	16	4.7934	0.000071 $[-0.000270, 0.000412]$	0.4968	9.57×10^{-6}
`s1_moe`	958,593	6142.5	16	4.7943	-0.000133 $[-0.000396, 0.000130]$	0.5159	9.47×10^{-6}
`s1_static`	958,593	6142.5	16	4.7943	-0.000134 $[-0.000393, 0.000125]$	0.5110	1.05×10^{-5}
`s2_m0`	363,897	2539.1	16	4.7934	0.000071 $[-0.000270, 0.000412]$	0.4968	9.57×10^{-6}
`s3_m0`	363,897	2539.1	16	4.7934	0.000071 $[-0.000270, 0.000412]$	0.4968	9.57×10^{-6}

Stage	Diagnostic result
S+1 MoE basis	`s1_moe` reported route-depth NMI 8.02×10^{-6}, AEBR = 1.385, and expert utilization 0.9992. `s1_static_moe` reported route-depth NMI 4.93×10^{-6}, AEBR = 1.512, and utilization 0.9995. Both are executable mechanism diagnostics, but neither improves the failed S+0 gate.
S+2 cache axis	The hidden-state proxy emitted $\alpha_{\mathrm{exact}} = 1.0$, $\alpha_{\mathrm{terminal}} = 0.0$, and $\alpha_{\mathrm{shared}} = 0.0$, with 2,097,152 bytes per terminal hidden state. The JSON label is `hidden_state_proxy_not_autoregressive_kv`; the quality gap is explicitly `not_tested_requires_autoregressive_terminal_context_attention`.
S+3 deployment	Fake-int6 linear-weight sensitivity reported best-loss gap +0.000907. The post-int6 $G_{\mathrm{nll}}(16,64)$ was 0.000044 with 95% CI $[-0.000295, 0.000383]$ and NDR = 0.5005, so quantization does not rescue depth scaling.

The outcome is the desired implementation split. All feedback stages execute under DDP on the requested GPU pair, but the S+0 positive-control gate still fails. Therefore the next scientific move remains detectability repair, not mechanism escalation.

11 Superseded All-Stage DDP Run

Before the final simplification, the full staged pipeline was run on June 4, 2026 with

`CUDA_VISIBLE_DEVICES=6,7 STAGES=all SEQ_LEN=8 ITERATIONS=1000 RUN_TAG=gpu67_allstages_seq8_i10`

That now-superseded runner completed and wrote eight stage outputs:

$$\{\texttt{s0\_control}, \texttt{s0\_m0}, \texttt{s0\_mclk}, \texttt{s1\_m0}, \texttt{s1\_moe}, \texttt{s1\_static\_moe}, \texttt{s2\_m0}, \texttt{s3\_m0}\}.$$

Run	Params	Peak MB	Best loss	$G_{\mathrm{nll}}(16,64)$ [95% CI]	$G_{\mathrm{acc}}(16,64)$ [95% CI]	NDR
`s0_control`	197,497	651.9	4.7896	0.000173 $[-0.000449, 0.000795]$	0.001465 $[-0.001781, 0.004710]$	0.4968
`s0_m0`	362,873	1244.5	4.7899	0.000000 $[-0.000392, 0.000392]$	0.000977 $[-0.000938, 0.002890]$	0.5002
`s0_mclk`	428,409	1246.1	4.7909	0.000027 $[-0.000331, 0.000384]$	0.000000 $[0.000000, 0.000000]$	0.4949
`s1_moe`	957,569	3095.6	4.7908	0.000112 $[-0.000228, 0.000453]$	-0.000732 $[-0.003027, 0.001563]$	0.5044
`s1_static`	956,545	3095.6	4.7914	0.000317 $[-0.000075, 0.000709]$	0.000488 $[-0.000865, 0.001842]$	0.4885

The longer, easier S+0 run still does not promote P1. The positive-control lower confidence bound is negative for both tested depth pairs and NDR_ϵ remains near 0.5. This is stronger evidence than the 300-iteration smoke: the bottleneck is not a missing MoE, quantizer, or cache feature. The next experimental move should be a detectability repair: either make the synthetic task expose an actually depth-dependent target to the current architecture, increase the training signal substantially, or add a simpler positive-control architecture that is known to solve the task before testing M_0.

This run is treated as evidence for deletion, not as an active run shape. S+1 code-path evidence was limited to executability: both `moe` and `static_moe` trained under DDP and reported expert utilization 1.0, but they did not improve the S+0 gate and are no longer options on the P1 runner. S+2 emitted the expected hidden-state proxy slopes:

$$\alpha_{\text{exact}} = 1.0, \qquad \alpha_{\text{terminal}} = 0.0, \qquad \alpha_{\text{shared}} = 0.0,$$

with 2,097,152 bytes per terminal state at the tested shape. This remains a synthetic proxy, not an autoregressive KV-cache proof. S+3 int6 evaluation ran; the post-int6 $G_{\text{nll}}(16, 64)$ for `s3_m0` was -0.000046 with 95% CI $[-0.000432, 0.000340]$, so deployment quantization also does not rescue depth scaling.

A Prior Diagnostic Evidence: 16x1 Dense-MoE Path

The previous in-repo path was a 16×1 dense-MoE recurrent model with Dirichlet–UCB routing, MLA-style expert attention, MoS output head, Parcae-style damping, int6 artifact scoring, and diagnostic K-sweeps. It is useful evidence because it failed P1 despite stable fixed-point diagnostics, motivating the final minimal design.

Metric	**Observed value**	**Interpretation**
Full validation BPB	3.702038	Diagnostic smoke score, not a promoted result.
$G_T(16, 64)$, $G_T(16, 128)$, $G_T(32, 128)$	-0.012196, -0.012865, -0.004076	Extra recurrence hurt paired task score.
NDR_ϵ	1.000000	Every tested pair degraded under the chosen threshold.
ρ_F, $\sigma_{\max}(J_F)$ at $K = 128$	0.7525, 0.9633	Stable/cache-ready diagnostics did not imply useful recurrence.
Route-depth NMI and utilization	near 0; near 0.99	Experts were broadly used but the mixture was nearly static across depth.
Peak VRAM	35632 MB	Memory feasible for the smoke profile.

K	BPB	ED_u	ED_ℓ	NMI mean	NMI max	util	ρ_F	$\sigma_{\max}$	iter rel.
4	3.7478	1.0484	1.0400	0.0001	0.0002	0.9944	0.7645	0.9853	0.1193
8	3.7556	1.1531	1.1738	0.0001	0.0001	0.9930	0.7564	0.9632	0.0317
16	3.7625	1.5920	2.3193	0.0001	0.0001	0.9919	0.7537	0.9623	0.0061
24	3.7655	2.7455	5.7373	0.0000	0.0001	0.9915	0.7530	0.9284	0.0027
32	3.7670	5.3277	10.0747	0.0000	0.0000	0.9913	0.7529	0.9533	0.0018
64	3.7687	24.0905	29.1213	0.0000	0.0000	0.9909	0.7528	0.9169	0.0014
128	3.7690	69.4624	62.6883	0.0000	0.0000	0.9908	0.7525	0.9633	0.0013
192	3.7690	113.3975	88.6915	0.0000	0.0000	0.9907	0.7526	1.0148	0.0013

The root-cause read is objective and measurement mismatch: the old stack contained many mechanisms, but no depth-hard positive control and no clean way to localize whether failure came from the harness, autonomous depth signal, block expressivity, or MoE routing. The final minimal protocol fixes that attribution problem before revisiting MoE or deployment features.

B Related Work

The mechanisms retained under the resource/expressiveness goals are each grounded in prior work; this section records the findings that justify them.

Reversible recurrence and reversible MoE. Reversible residual coupling [**?**] reconstructs layer inputs from outputs, removing activation storage; Reformer [**?**] applies it to Transformers. RevFFN [**?**] combines reversible two-stream blocks with a Mixture-of-Experts FFN for memory-efficient full-parameter MoE training (≈49% peak-VRAM reduction, *no* floor/contraction requirement), directly validating the reversible×MoE combination used here. Our additive coupling (§**??**) drives each update from the *opposite* stream, yielding an exact *explicit* inverse (no fixed-point iteration, unlike RevFFN's cross-attention coupling).

Recurrent depth, weight tying, and expressivity recovery. Universal Transformers [**?**], Huginn [**?**], and Parcae [**?**] treat recurrent depth as a parameter-saving axis. Weight tying loses per-layer diversity, and sparse MoE recovers it: MoEUT [**?**] shows MoE capacity compensates for UT-style layer sharing, Sparse-Looped-MoE [**?**] recovers functional diversity across loop passes, and Relaxed Recursive Transformers [**?**] relax tied layers with layer-wise LoRA. This is exactly our mechanism: low-rank experts shared across recurrence steps, composed per step into high *effective* depth.

Smooth, reversibility-safe sparse routing. Hard top-k routing is non-smooth and reconstruction-unstable under reversibility. ReMoE's ReLU routing [**?**] is continuous and fully differentiable yet yields exact-zero (sparse) gates, with adaptive-L_1 sparsity control and load balancing; it matches top-k FLOPs, exceeds top-k accuracy, and shows 2–3× lower activation flip-rate (hence more reconstruction-stable). We expose `router_type`$\in$ {`relu`, `softmax`} as a control experiment.

Effective-depth and rank metrics. The recurrence-equivalence exponent φ [**?**] measures whether looping a block r times yields the capacity of r unique blocks (φ=1) or none (φ=0), separating true loop capacity from token-budget effects; truncated BPTT lowers φ, motivating our full reversible BPTT. Effective rank (spectral entropy of singular values) [**?**] sets each low-rank matrix's "sufficient rank" on the resource/expressiveness frontier.

Attention and output expressiveness under a resource budget. Multi-head Latent Attention [**?**] low-rank-compresses the KV cache (to 4–14% of MHA), the basis for the MLA block. The Mixture-of-Softmaxes head [**?**] breaks the softmax-rank bottleneck for output-distribution expressiveness; at vocabulary size 1024 its per-component cost is small.

References

[1] D. A. M. Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1. *Journal of Computer and System Sciences*, 38(1):150–164, 1989. https://doi.org/10.1016/0022-0000(89)90037-8.

[2] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal Transformers. *International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1807.03819.

[3] J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171, 2025. https://arxiv.org/abs/2502.05171.

[4] K. Schwethelm, D. Rueckert, and G. Kaissis. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. arXiv:2604.21106, 2026. https://arxiv.org/abs/2604.21106.

[5] V. C. Vendrell, A. P. Masdemont, N. Grillo, J. Ros-Giralt, A. Behboodi, and F. V. Massoli. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. arXiv:2605.07721, 2026. https://arxiv.org/abs/2605.07721.

[6] R. Csordas, K. Irie, J. Schmidhuber, C. Potts, and C. D. Manning. MoEUT: Mixture-of-Experts Universal Transformers. *NeurIPS*, 2024. https://arxiv.org/abs/2405.16039.

[7] A. N. Gomez, M. Ren, R. Urtasun, and R. B. Grosse. The Reversible Residual Network: Backpropagation Without Storing Activations. *NeurIPS*, 2017. https://arxiv.org/abs/1707.04585.

[8] N. Kitaev, Ł. Kaiser, and A. Levskaya. Reformer: The Efficient Transformer. *International Conference on Learning Representations*, 2020. https://arxiv.org/abs/2001.04451.

[9] RevFFN: Memory-Efficient Full-Parameter Fine-Tuning of Mixture-of-Experts LLMs with Reversible Blocks. arXiv:2512.20920, 2025. https://arxiv.org/abs/2512.20920.

[10] Z. Wang, J. Chen, and J. Zhu. ReMoE: Fully Differentiable Mixture-of-Experts with ReLU Routing. *International Conference on Learning Representations*, 2025. https://arxiv.org/abs/2412.14711.

[11] S. Bae, A. Fisch, H. Harutyunyan, Z. Ji, S. Kim, and T. Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. arXiv:2410.20672, 2024. https://arxiv.org/abs/2410.20672.

[12] Sparse Looped-MoE: Recovering Functional Diversity Across Loop Passes. arXiv:2605.09165, 2026. https://arxiv.org/abs/2605.09165.

[13] Parcae: Stable Looped Language Models with Loop Depth as a Scaling Axis. arXiv:2604.12946, 2026. https://arxiv.org/abs/2604.12946.

[14] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024. https://arxiv.org/abs/2405.04434.

[15] Z. Yang, Z. Dai, R. Salakhutdinov, and W. W. Cohen. Breaking the Softmax Bottleneck: A High-Rank RNN Language Model. *International Conference on Learning Representations*, 2018. https://arxiv.org/abs/1711.03953.

[16] O. Roy and M. Vetterli. The Effective Rank: A Measure of Effective Dimensionality. *European Signal Processing Conference (EUSIPCO)*, 2007.